Automatic differentiation in QAOA

https://github.com/qiboteam/qibo/issues/597

ROW 129

Automatic differentiation in QAOA #597

New issue

Closed

AdrianPerezSalinas opened on Jun 8, 2022

Hi all,

I am trying to run an automatic differentiation in QAOA. It turns out that the gradients are for some reason unsupported by tensorflow in the case where the algorithm is run using the exponential solver.

The code I am using is

```
import numpy as np
from qibo import models, hamiltonians
import tensorflow as tf
from qibo import set_backend

set_backend('tensorflow')

# Create XXZ Hamiltonian for six qubits
hamiltonian = hamiltonians.XXZ(6)
# Create QAOA model
qaoa = models.QAOA(hamiltonian)

optimizer = tf.keras.optimizers.Adam()

# Optimize starting from a random guess for the variational parameters
params = 0.01 * tf.Variable(tf.random.uniform((4,), dtype=tf.float64))
nepochs = 1000


for _ in range(nepochs):
    with tf.GradientTape() as tape:
        qaoa.set_parameters(params)
        final_state = qaoa.execute().state()
        energy = hamiltonian.expectation(final_state)

    grads = tape.gradient(energy, params)
    optimizer.apply_gradients(zip([grads], [params]))
```

and the error I get is

```
[Qibo 0.1.7|INFO|2022-06-08 15:00:03]: Using tensorflow backend on /device:CPU:0
Traceback (most recent call last):
  File "/media/adrianps/7CAE-EA93/AQA/depth-limited-quantum-computation/APS/sandbox.py", line 23, in <module>
    final_state = qaoa.execute().state()
  File "/home/adrianps/venvs/depth_limit/lib/python3.8/site-packages/qibo/models/variational.py", line 344, ir
    state = self._apply_exp(state, self.ham_solver,
  File "/home/adrianps/venvs/depth_limit/lib/python3.8/site-packages/qibo/models/variational.py", line 325, ir
    state = solver(state)
  File "/home/adrianps/venvs/depth_limit/lib/python3.8/site-packages/qibo/solvers.py", line 84, in __call__
    propagator = self.current_hamiltonian.exp(self.dt)
  File "/home/adrianps/venvs/depth_limit/lib/python3.8/site-packages/qibo/core/hamiltonians.py", line 63, in e
    self._exp["result"] = self.K.expm(-1j * a * self.matrix)
  File "/home/adrianps/venvs/depth_limit/lib/python3.8/site-packages/tensorflow/python/util/traceback_utils.py
    raise e.with_traceback(filtered_tb) from None
  File "/home/adrianps/venvs/depth_limit/lib/python3.8/site-packages/tensorflow/python/framework/constant_op.p
    return ops.EagerTensor(value, ctx.device_name, dtype)
TypeError: Cannot convert (-0-1j) to EagerTensor of dtype double
```

Can you please share any insight about what it the problem here?

Thanks!

Assignees

No one assigned

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

AdrianPerezSalinas added bug on Jun 8, 2022

andrea-pasquale on Jun 9, 2022 · edited by andrea-pasquale — Edits — Contributor

If I understand correctly what you were doing I believe that the best solution is just to use directly the `minimize` method of the QAOA object. You can even set the optimizer and the number of epochs that will be used by passing a dictionary containing the key "optimizer" and the key "nepochs" as an argument. The following code should match exactly what you were doing, using the `Adam` optimizer.

```
import numpy as np
from qibo import models, hamiltonians, set_backend
set_backend("tensorflow")
# create XXZ Hamiltonian for four qubits
hamiltonian = hamiltonians.XXZ(6)
# create QAOA model for this Hamiltonian
qaoa = models.QAOA(hamiltonian)

initial_parameters = 0.01 * np.random.uniform(size=4)
options = {}
options["optimizer"] = "Adam"
options["nepochs"] = 1000
best_energy, final_parameters, extra = qaoa.minimize(initial_parameters, method="sgd", options=options)
```

If you prefer to code it explicitly you need to be careful about the type of variables that you are using when dealing with tf optimization. For example in this line

> params = 0.01 * tf.Variable(tf.random.uniform((4,), dtype=tf.float64))

you are implicitly converting a tf.Variable into a Tensor:

```
print(type(0.01 * tf.Variable(tf.random.uniform((4,), dtype=tf.float64))))
# output
# <class 'tensorflow.python.framework.ops.EagerTensor'>
```

Also some qibo methods requires specific types, such as `set_parameters` in this particular case.
For your particular problem this is the correct version of the code:

```
import numpy as np
from qibo import models, hamiltonians
import tensorflow as tf
from qibo import set_backend, K

set_backend('tensorflow')

# Create XXZ Hamiltonian for six qubits
hamiltonian = hamiltonians.XXZ(6)
# Create QAOA model
qaoa = models.QAOA(hamiltonian)

optimizer = tf.keras.optimizers.Adam()
# Optimize starting from a random guess for the variational parameters
params = tf.Variable(0.01 * tf.random.uniform((4,), dtype=tf.float64))
nepochs = 1000


for _ in range(nepochs):
    with tf.GradientTape() as tape:
        qaoa.set_parameters(K.cast(params))
        final_state = qaoa.execute()
        energy = hamiltonian.expectation(final_state)

    grads = tape.gradient(energy, [params])
    optimizer.apply_gradients(zip(grads, [params]))
```

AdrianPerezSalinas on Jun 9, 2022 — Contributor — Author

Thank you very much for the answer, this works fine.

I wanted to add, however, that I am trying to optimize a circuit in this way because it is part of a much more complicated model.

Also some qibo methods requires specific types, such as `set_parameters` in this particular case.
For your particular problem this is the correct version of the code:

```
import numpy as np
from qibo import models, hamiltonians
import tensorflow as tf
from qibo import set_backend, K

set_backend('tensorflow')

# Create XXZ Hamiltonian for six qubits
hamiltonian = hamiltonians.XXZ(6)
# Create QAOA model
qaoa = models.QAOA(hamiltonian)

optimizer = tf.keras.optimizers.Adam()
# Optimize starting from a random guess for the variational parameters
params = tf.Variable(0.01 * tf.random.uniform((4,), dtype=tf.float64))
nepochs = 1000


for _ in range(nepochs):
    with tf.GradientTape() as tape:
        qaoa.set_parameters(K.cast(params))
        final_state = qaoa.execute()
        energy = hamiltonian.expectation(final_state)

    grads = tape.gradient(energy, [params])
    optimizer.apply_gradients(zip(grads, [params]))
```

AdrianPerezSalinas on Jun 9, 2022 — Contributor — Author

Thank you very much for the answer, this works fine.

I wanted to add, however, that I am trying to optimize a circuit in this way because it is part of a much more complicated model. I will probably have some more problems as I keep going, so I prefer not to close the issue until a little bit more of work is done.

Thanks!!

scarrazza closed this as completed on Jul 11, 2022